

DELIVERABLE 1: PRODUCTION SQL QUERIES

Data Vortex Competition — Aaruush '26, Round 1, Phase 2

Selected: E2 (Easy) | M5 (Medium) | H5 (Hard)

Schema Reference

```
Table: users (user_id, location, language, followers)
```

```
Table: posts (post_id, user_id, platform, text_content, likes, shares, comments)
```

E2: Most Engaged Posts (EASY)

Find the top 10 posts based on likes + shares + comments. Ignore posts with missing likes.

```

-- =====
-- E2: MOST ENGAGED POSTS
-- =====
-- GOAL: Identify the top 10 most engaged posts by total engagement
--       score (likes + shares + comments). Posts with NULL likes are
--       excluded since their engagement score is indeterminate.
--
-- TECHNIQUE: CTE pipeline + DENSE_RANK window function
-- ANSI COMPLIANCE: Standard SQL, no vendor-specific extensions
-- =====

```

```

WITH engagement_scored AS (
    -- Step 1: Compute total engagement for each post,
    --       filtering out records where likes data is missing.
    SELECT
        p.post_id,
        p.user_id,
        p.platform,
        SUBSTR(p.text_content, 1, 80)      AS text_preview,
        CAST(p.likes AS INTEGER)           AS likes,
        p.shares,
        p.comments,
        -- Core engagement formula: sum of all three interaction metrics
        (CAST(p.likes AS INTEGER) + p.shares + p.comments)
                                           AS total_engagement,
        u.location,
        u.language,
        u.follower_count
    FROM posts p
    JOIN users u ON p.user_id = u.user_id
    WHERE p.likes IS NOT NULL -- Exclude posts with missing likes
),
ranked_posts AS (
    -- Step 2: Rank posts by total engagement using a window function.
    --       DENSE_RANK ensures tied engagement scores share the same
    rank.
    SELECT

```

```

        *,
        DENSE_RANK() OVER (
            ORDER BY total_engagement DESC
        ) AS engagement_rank
    FROM engagement_scored
)
-- Step 3: Return only the top 10 most engaged posts.
SELECT
    engagement_rank,
    post_id,
    user_id,
    platform,
    text_preview,
    likes,
    shares,
    comments,
    total_engagement,
    location,
    follower_count
FROM ranked_posts
WHERE engagement_rank <= 10
ORDER BY engagement_rank ASC, total_engagement DESC;

```

M5: Detect Suspicious Engagement (MEDIUM)

Find posts where shares > likes + comments combined. Return top 20 sorted by shares descending.

```
-- =====
-- M5: DETECT SUSPICIOUS ENGAGEMENT
-- =====
-- GOAL: Flag posts exhibiting suspicious engagement patterns where
--       shares exceed the combined total of likes and comments.
--       This is a classic bot/manipulation indicator: organic posts
--       rarely have more shares than likes+comments combined.
--
-- TECHNIQUE: CTE with computed inequality filter + ratio analysis
-- WINDOW FUNCTION: PERCENT_RANK to show how extreme each case is
-- =====
```

```
WITH suspicious_posts AS (
  -- Step 1: Identify posts where shares exceed likes + comments.
  --       Only consider posts with non-null likes for fair
comparison.
  SELECT
    p.post_id,
    p.user_id,
    p.platform,
    SUBSTR(p.text_content, 1, 60)      AS text_preview,
    CAST(p.likes AS INTEGER)           AS likes,
    p.shares,
    p.comments,
    (CAST(p.likes AS INTEGER) + p.comments)
                                     AS likes_plus_comments,
    -- Share surplus: how much shares exceed likes+comments
    (p.shares - (CAST(p.likes AS INTEGER) + p.comments))
                                     AS share_surplus,
    -- Share-to-engagement ratio: proportion of total from shares
    ROUND(
      CAST(p.shares AS REAL) /
      NULLIF(CAST(p.likes AS INTEGER) + p.shares + p.comments, 0)
      * 100, 1
    )
                                     AS share_pct_of_total,
    u.location,
    u.follower_count
```

```

FROM posts p
JOIN users u ON p.user_id = u.user_id
WHERE p.likes IS NOT NULL
      AND p.shares > (CAST(p.likes AS INTEGER) + p.comments)
),
ranked_suspicious AS (
  -- Step 2: Rank suspicious posts by share volume and compute
  --          percentile to measure how extreme the anomaly is.
  SELECT
    *,
    ROW_NUMBER() OVER (
      ORDER BY shares DESC
    ) AS suspicion_rank,
    ROUND(
      PERCENT_RANK() OVER (
        ORDER BY share_surplus ASC
      ) * 100, 1
    ) AS surplus_percentile
  FROM suspicious_posts
)
-- Step 3: Return the top 20 most suspicious posts.
SELECT
  suspicion_rank,
  post_id,
  user_id,
  platform,
  text_preview,
  likes,
  shares,
  comments,
  likes_plus_comments,
  share_surplus,
  share_pct_of_total,
  location,
  follower_count
FROM ranked_suspicious

```

```
WHERE suspicion_rank <= 20  
ORDER BY suspicion_rank ASC;
```

H5: Identify Data Anomalies (HARD)

Identify corrupted records matching: negative likes, missing platforms, missing text, or text containing HTML entities/tags.

```

-- =====
-- H5: IDENTIFY DATA ANOMALIES
-- =====
-- GOAL: Perform a comprehensive data integrity audit by identifying
--       all corrupted records across four anomaly categories:
--       1. Negative likes (sign corruption in numeric fields)
--       2. Missing platform metadata (NULL or empty)
--       3. Missing text content (NULL, empty, or literal 'NULL')
--       4. HTML contamination (tags like <br>, <div> or entities
--          like & embedded in text fields)
--
-- TECHNIQUE: UNION ALL of four independent anomaly scans, each
--            tagged with its anomaly type, combined with window
--            functions for per-category statistics.
-- NOTE: This query runs against the RAW corrupted source data
--       to detect all original anomalies before cleaning.
-- =====

```

```

WITH anomaly_scan AS (

```

```

    -- Category 1: NEGATIVE LIKES

```

```

    -- Engagement metrics should never be negative. Negative values
    -- indicate sign-corruption during data transfer or ETL failure.

```

```

    SELECT

```

```

        post_id, user_id, platform,
        SUBSTR(text_content, 1, 50)      AS text_preview,
        likes                            AS flagged_value,
        'NEGATIVE_LIKES'                 AS anomaly_type,
        'Engagement metric has impossible negative value'
                                          AS anomaly_description

```

```

    FROM posts_raw

```

```

    WHERE CAST(likes AS REAL) < 0

```

```

    UNION ALL

```

```

    -- Category 2: MISSING PLATFORM

```

```

    -- Platform field is NULL, empty, or contains the literal 'NULL'.

```

```

    SELECT

```

```

        post_id, user_id, platform,
        SUBSTR(text_content, 1, 50)           AS text_preview,
        platform                             AS flagged_value,
        'MISSING_PLATFORM'                   AS anomaly_type,
        'Platform metadata is absent or null-string'
                                                AS anomaly_description
FROM posts_raw
WHERE platform IS NULL
    OR TRIM(platform) = ''
    OR TRIM(UPPER(platform)) = 'NULL'

UNION ALL

-- Category 3: MISSING TEXT CONTENT
-- Text content is NULL, empty, or the literal string 'NULL'.
SELECT
    post_id, user_id, platform,
    SUBSTR(text_content, 1, 50)           AS text_preview,
    CASE
        WHEN text_content IS NULL THEN '<NULL>'
        WHEN TRIM(text_content) = '' THEN '<EMPTY>'
        ELSE 'NULL-string'
    END                                   AS flagged_value,
    'MISSING_TEXT'                         AS anomaly_type,
    'Post body is absent, empty, or null-string'
                                                AS anomaly_description
FROM posts_raw
WHERE text_content IS NULL
    OR TRIM(text_content) = ''
    OR TRIM(UPPER(text_content)) = 'NULL'

UNION ALL

-- Category 4: HTML CONTAMINATION
-- Text contains raw HTML tags or unescaped HTML entities.
SELECT
    post_id, user_id, platform,

```

```

        SUBSTR(text_content, 1, 50)                AS text_preview,
        CASE
            WHEN text_content LIKE '%<br>%'        THEN '<br> tag found'
            WHEN text_content LIKE '%<div>%'        THEN '<div> tag found'
            WHEN text_content LIKE '%</div>%'        THEN '</div> tag found'
            WHEN text_content LIKE '%&%'            THEN '& entity found'
            WHEN text_content LIKE '%&lt;%'          THEN '&lt; entity found'
            WHEN text_content LIKE '%&gt;%'          THEN '&gt; entity found'
            ELSE 'Other HTML artifact'
        END                                         AS flagged_value,
        'HTML_CONTAMINATION'                       AS anomaly_type,
        'Text contains raw HTML tags or unescaped entities'
                                                    AS anomaly_description

FROM posts_raw
WHERE text_content LIKE '%<br>%'
    OR text_content LIKE '%<div>%'
    OR text_content LIKE '%</div>%'
    OR text_content LIKE '%&%'
    OR text_content LIKE '%&lt;%'
    OR text_content LIKE '%&gt;%'
),
anomaly_summary AS (
    SELECT
        *,
        COUNT(*) OVER (PARTITION BY anomaly_type) AS category_total,
        ROW_NUMBER() OVER (
            PARTITION BY anomaly_type ORDER BY post_id
        )                                         AS row_within_category
    FROM anomaly_scan
)
SELECT
    anomaly_type,
    category_total,
    row_within_category,
    post_id,
    user_id,
    platform,

```

```

        text_preview,
        flagged_value,
        anomaly_description
FROM anomaly_summary
WHERE row_within_category <= 5
ORDER BY anomaly_type, row_within_category;

```

```

-- =====
-- H5 SUPPLEMENTARY: ANOMALY SUMMARY AGGREGATION
-- =====

```

```

SELECT
    anomaly_type,
    COUNT(*)                                AS record_count,
    COUNT(DISTINCT user_id)                 AS affected_users,
    ROUND(
        COUNT(*) * 100.0 /
        (SELECT COUNT(*) FROM posts_raw), 2
    )                                        AS pct_of_total
FROM (
    SELECT post_id, user_id, 'NEGATIVE_LIKES' AS anomaly_type
    FROM posts_raw WHERE CAST(likes AS REAL) < 0
    UNION ALL
    SELECT post_id, user_id, 'MISSING_PLATFORM'
    FROM posts_raw
    WHERE platform IS NULL OR TRIM(platform) = '' OR
    TRIM(UPPER(platform)) = 'NULL'
    UNION ALL
    SELECT post_id, user_id, 'MISSING_TEXT'
    FROM posts_raw
    WHERE text_content IS NULL OR TRIM(text_content) = '' OR
    TRIM(UPPER(text_content)) = 'NULL'
    UNION ALL
    SELECT post_id, user_id, 'HTML_CONTAMINATION'
    FROM posts_raw
    WHERE text_content LIKE '%<br>%'

```

```
        OR text_content LIKE '%<div>%' OR text_content LIKE '%</div>%'
        OR text_content LIKE '%&%' OR text_content LIKE '%&lt;%' OR
text_content LIKE '%&gt;%'
) anomalies
GROUP BY anomaly_type
ORDER BY record_count DESC;
```